PROJECT GUIDE / 01

Northbridge KYC

PROJECT REPORT & LOCAL REVIEW GUIDE

Document consistency. Local extraction. Human decisions.

A full-stack application for reviewing applicant identity and payroll documents. It compares documentary names and salary-slip PAN numbers, then requires an authorized staff decision.

Applicant submits	System evaluates	Staff decides
Aadhaar + PAN + one or more salary slips. Optional bank statement.	Names agree across documents; each salary-slip PAN equals the uploaded PAN.	Approve, reject, or request corrections after reviewing evidence.

Purpose and audience

This guide is for a GitHub reviewer or developer who needs to understand the implemented features, run the application locally, inspect the architecture and evaluate the current behavior.

Important scope

Approval confirms document consistency only. It is not government authentication, a guarantee that documents are genuine, or a loan/credit decision. Synthetic format warnings are informational; untrusted sample identifiers are not promoted into verified identity records.

Prepared from the repository on 11 November 2026. README.md is the primary runnable setup guide. The report contains no local secrets, real applicant documents or administrator credentials.

Features and roles

Role	Implemented capabilities
Applicant	Register/sign in; verification and password reset; mandatory Aadhaar/PAN uploads; salary slips; optional bank statement; page previews; replacement/removal; consent; versioned corrections; simple results; approval/rejection stamps; notifications.
Reviewer	Organization-scoped queue; assignment and release; live processing stages; masked evidence; authorized reveals/originals; name/PAN comparison; internal notes; approval, rejection and correction requests.
Administrator	Full applicant profile within authorized organizations; direct privileged document access; reviewer features; staff management; policy limits; operational metrics; audited identity conflict resolution.

Recent additions reflected in the code

- Mandatory PAN upload and salary-slip PAN-number equality checks.
- Applicant document previews before submission, including multiple pages and original files.
- Staff document inventory with current stage, errors, evidence boxes and exact-field reveals.
- Applicant match summaries without exposing detailed technical processing stages.
- KYC completed-and-approved UI, shared decision stamps and in-app notifications.
- Dynamic overview counts aligned with queue filters; five-second polling for current views.
- Timezone-aware persistence/display; rejection after resubmission handles retention dates.
- Column-aware OCR and checks that prevent identifiers being accepted as names.

End-to-end workflow

1. Prepare account

Applicant registers, verifies email and opens a saved draft.

2. Upload and review

Aadhaar, PAN and salary slips are uploaded. Each file can be previewed before consent/submission.

3. Queue processing

API creates a durable process_version job. The separate worker claims it.

4. Extract safely

Local OCR/text extraction, masking and an outbound privacy gate precede the Gemini call.

5. Validate and compare

The response must satisfy schema, token-type and line-evidence rules. Local rules compare names and PAN numbers.

6. Human review

Staff inspect the originals, extracted fields, checks and warnings, then record a decision.

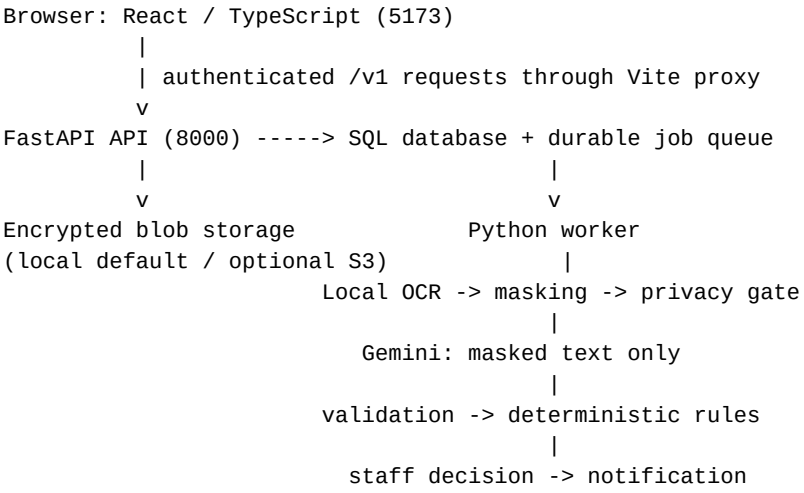
7. Notify applicant

Applicant sees the result, decision stamp and in-app notification. Corrections produce a new version.

Versioning and reprocessing

Submitted versions are immutable to the applicant. Starting a correction creates a new current draft and supersedes the previous version. Worker attempts are versioned; old results do not silently become the latest result after a code change. Staff can reprocess eligible undecided submissions. Approval uses version/row-version checks to reject stale decisions.

Architecture and runtime services



Three processes are intentional

The API handles HTTP requests, authentication and submission. The worker performs document processing outside request handlers. Vite serves the frontend and proxies API requests. Starting the UI and API without the worker leaves documents queued.

Persistence and communication

SQLite is an explicit local-development choice and supports one worker. PostgreSQL is the intended concurrent-worker database. Processing events support authenticated server-sent events with polling fallback. The overview/queue and applicant views also poll for current data. File contents are encrypted in private storage, not exposed as static URLs.

Alternative deployment

Docker Compose defines PostgreSQL, one-shot migration, API, worker and Nginx/web services. It serves the application at port 8080. Review README.md before using external PostgreSQL with Compose: backend env_file values are distinct from Compose interpolation variables.

OCR, NLP and AI models

Component	Model or mechanism	Responsibility
Embedded PDF text	PyMuPDF	Read text and page/word geometry locally; no OCR confidence is invented.
Image/scanned PDF OCR	Tesseract, eng; OpenCV/Pillow preprocessing	EXIF handling, grayscale/deskew/upscale and bounded alternate OCR when needed.
Personal-data detection	Microsoft Presidio + spaCy en_core_web_sm 3.8.0 + custom rules	Recognize names, labels, Aadhaar/PAN/account/contact details; retain coordinates.
Tokenization	Application-attempt TokenVault	Replace values locally with typed placeholders. Normalized equal names/PAN values reuse tokens.
Structured extraction	google-genai; configured Gemini model	Classify documents and copy tokens/amounts with evidence line numbers. Default: gemini-3.5-flash-lite.
Validation	Pydantic plus local evidence validation	Reject invented, mistyped, absent or incorrectly cited values.
Comparison	Deterministic Python rules; RapidFuzz hints	Determine consistency; never delegate match/approval decisions to the LLM.

Outbound boundary

Only a minimized, privacy-gated text payload is sent to Gemini. Raw files, the token vault and original personal values stay in the backend. An uncertain payload is withheld for manual review. Gemini access still requires a supported model, API key, network connectivity and available quota.

Test model

ScriptedProvider is a clearly identified test double, allowed only in APP_ENV=test. It supports reproducible tests without Gemini and is refused as a normal/production fallback. The spaCy model is installed during dependency setup; no NLP model is fetched during a request.

Matching rules and outcomes

- **Mandatory:** Aadhaar, PAN and the configured minimum number of readable salary slips.

- **Name agreement:** PAN holder name equals Aadhaar holder name; each salary slip employee name equals both.

- **PAN agreement:** each distinct salary slip contains a readable PAN value equal to the uploaded PAN card number.

- **Normalization:** case/spacing/punctuation for names; canonical PAN values. Fuzzy/reordered-name hints do not automatically match.

- **Excluded criteria:** father name is not compared; registration name and bank statements are informational.

Scenario	Result
Same documentary names; same printed salary-slip PAN	Match; staff approval still required.
Salary name differs from Aadhaar or PAN	Mismatch, with the affected name comparison.
Salary-slip PAN differs from uploaded PAN	Mismatch, with PAN-number explanation.
Salary slip has no readable PAN, or required name is unavailable	Incomplete; request clearer/corrected evidence or manual review.
Matching synthetic PAN values fail format plausibility	Consistency can still match. Identifier warning is informational; no verified identity reservation.
Unresolved cross-account identity conflict	Staff warning and approval blocker; administrator resolution required.

Avoiding false name mismatches

Local extraction splits spatially distant columns so QR-code OCR noise is not appended to a holder name. Identifier-shaped values cannot be treated as person names. Salary employee names need a supported label association; uncertain or ambiguous selection is routed to review instead of inventing a match.

Data models, privacy and access

Entity group	Models and purpose
Organization/access	Organization, OrgPolicy, User, StaffMembership: ownership, roles and configurable limits; PAN remains mandatory.
Authentication	AuthSession, AuthToken, LoginAttempt: cookie sessions, verification/reset tokens and login protection.
Applications	Application, ApplicationVersion: references, current/superseded versions, processing/review/check status and concurrency fields.
Documents/evidence	Document, ExtractionResult, VerificationCheck: storage references, per-page metadata, encrypted exact fields, masked display, provenance and comparison outcomes.
Jobs/progress	ProcessingJob, ProcessingEvent: durable jobs, attempts, leases, cancellation and stage events.
Identity/review	IdentityClaim, ConfirmedIdentity, Review: duplicate conflicts, valid identity reservations and audited human decisions. Untrusted samples remain unvalidated.
Communication/audit	Notification, EmailOutbox, AuditEvent: in-app messages, development/SMTP mail and value-minimized access/action history.

Security implementation

Passwords use Argon2. Exact extracted values and document blobs use application-level AES-GCM encryption. Identity fingerprints use a separate keyed HMAC. Access is checked by applicant ownership or staff organization membership. State-changing requests use CSRF protection with cookie sessions. Original/reveal access is privileged and audited; normal evidence views are masked.

Retention and timestamps

Originals are scheduled for deletion after a final decision (default seven days) and removed by retention jobs. Encrypted extraction records remain in the database. Backup deletion is outside the application. UTC timestamps are normalized at the database boundary and displayed in the browser local timezone.

This is an implemented prototype, not a certified security or regulatory assessment. No credentials or applicant files should be committed to GitHub.

Local setup: dependencies and configuration

Use README.md for complete OS-specific commands. Run backend commands from backend and frontend commands from frontend. Python must be 3.12; Node 22 or 24 and npm are expected.

```
# From repository root
cd backend
uv python install 3.12
uv sync --locked
cd ../frontend
npm ci
cd ..
```

System requirements

Git, uv, Python 3.12, Node/npm, Tesseract with eng language data, a Gemini key and a writable database/storage location. SQLite is enough for local review. PostgreSQL, Docker/Compose, Playwright browsers and S3/boto3 are optional according to the selected workflow.

Create a private local environment file

Copy backend/.env.example to backend/.env once. Create backend/var before opening a relative SQLite path. Do not overwrite an existing configured environment file.

```
APP_ENV=development
PUBLIC_BASE_URL=http://localhost:5173
ALLOWED_ORIGINS=["http://localhost:5173"]
DATABASE_URL=sqlite:///./var/dev.db
MIGRATION_DATABASE_URL=
ALLOW_SQLITE=true
GEMINI_API_KEY=YOUR_OWN_KEY
GEMINI_MODEL=gemini-3.5-flash-lite
TESSERACT_CMD=
TESSERACT_LANG=eng
STORAGE_BACKEND=local
STORAGE_DIR=./var/storage
PAN_POLICY=required
```

Leave TESSERACT_CMD blank if it is on PATH; otherwise set its executable path. Keep remaining limits from the example. In the backend directory, run gen-secrets and paste its three generated values into .env. The command prints secrets; it does not save them.

Database, administrator and three terminals

Initialize once, inside backend

```
uv run python -m app.cli gen-secrets
# Paste SESSION_SECRET, ENCRYPTION_KEY and IDENTITY_HMAC_SECRET into .env
uv run python -m app.cli migrate
uv run python -m app.cli preflight
uv run python -m app.cli bootstrap-admin --email "you@example.com" --name "Reviewer"
```

Use your own email/password. Run migrate before preflight on a fresh database because preflight checks migration status. Preserve encryption/HMAC secrets across restarts. The administrator is stored in the database; it is not hardcoded in the source.

Terminal 1: API (from repository root)

```
cd backend
uv run uvicorn app.main:app --port 8000 --reload
```

Terminal 2: worker (from repository root)

```
cd backend
uv run python -m app.cli worker
```

Terminal 3: frontend (from repository root)

```
cd frontend
npm run dev
```

URL	Use
http://localhost:5173	Applicant portal
http://localhost:5173/staff/login	Reviewer/admin portal
http://localhost:8000/docs	FastAPI API documentation
http://localhost:8000/health	API health check
http://localhost:5173/dev/mailbox	Development mail when SMTP is absent

Reviewer walkthrough and API map

Recommended local review

- Create a separate applicant account. Open the development mailbox to verify its email.

- Upload clearly synthetic Aadhaar, PAN and salary documents with identical names and the same PAN printed on the salary slip. Preview all pages before submitting.

- Watch the staff application timeline, inspect exact values and name/PAN comparisons, then approve.

- Confirm the applicant receives the approved banner, decision stamp and in-app notification.

- Change a salary name or PAN for a mismatch case; omit a printed salary PAN for an incomplete case.

- Request corrections or reject with an explanatory message. A correction produces a new submission version.

Endpoint families

Routes	Purpose
/v1/auth/*	Registration, sessions, verification and password-reset flows.
/v1/applications/*	Applicant draft, upload, submit, revise, cancel and progress operations.
/v1/documents/{id}/pages/{page}	Authorized page previews; reveal=true requires ownership/privilege.
/v1/documents/{id}/original and /reveal	Authorized originals and exact extracted fields.
/v1/staff/dashboard and /applications/*	Overview, queue, details, assignment, decisions and reprocessing.
/v1/admin/*	Staff, policies, metrics and identity-conflict administration.
/v1/notifications/*	Applicant/staff in-app notifications and read state.

Interactive endpoint contracts are available at /docs, with a repository snapshot in docs/openapi.json. Browser clients must maintain the session cookie and CSRF token for state-changing requests.

Verification, limitations and troubleshooting

How to verify locally

```
# backend directory: isolated test DBs, scripted extraction by default
uv run pytest -q
# Known Pillow/DPI issue can be excluded while it is investigated:
uv run pytest -q -k "not skewed_and_exif_rotated_image"

# frontend directory
npm test
npm run typecheck
npm run build
npm run lint
```

Backend tests cover document validation, masking/privacy gates, evidence validation, matching/PAN requirements, authorization, review actions, queues, notifications, retention and timestamps. Frontend tests cover result states, decisions, previews, forms, timeline behavior and time formatting. PostgreSQL tests skip without a dedicated test database; live Gemini is opt-in.

Latest recorded verification - 11 November 2026

Backend filtered run: 104 passed, 6 skipped, 2 deselected. Frontend: 24 tests passed and TypeScript checking passed. The known image-DPI case was excluded, live Gemini remains opt-in and PostgreSQL tests require a separate test database.

What is and is not verified

Recent local work exercised backend regression tests with isolated SQLite, Tesseract OCR cases, frontend tests/type checking and production builds. A synthetic-placeholder Gemini request verified the corrected response schema. This report does not claim that every uploaded document or Gemini model response has been independently verified. Docker, fresh Windows setup, S3/SMTP infrastructure and live PostgreSQL were not re-executed for this report.

Known limitations: one image-DPI metadata test is environment-sensitive and currently excluded in filtered runs; frontend lint has Fast Refresh warnings; backend Ruff has existing style findings. OCR and model outputs can still fail validation and require review. Existing sample files may predate the PAN-in-salary rule; regeneration is explicit, not automatic.

Common setup failures

No module named app: run inside backend. **SQLite cannot open:** create backend/var and use the correct working directory. **Queued forever:** start the worker. **Origins parsing:** use a JSON array. **Model/key error:** check Google access, quota and worker logs. **Awaiting review:** a human decision is required. **Old result:** restart the worker and reprocess eligible versions.

Dependency inventory: backend

These are direct constraints from pyproject.toml, not claims about the newest package release. uv.lock contains exact resolved and transitive packages. uv sync --locked installs both runtime and dev groups, including the spaCy model.

Backend runtime

Dependency / constraint	Installation
fastapi>=0.115	Installed by uv sync --locked
uvicorn[standard]>=0.30	Installed by uv sync --locked
pydantic>=2.8	Installed by uv sync --locked
pydantic-settings>=2.4	Installed by uv sync --locked
email-validator>=2.2	Installed by uv sync --locked
sqlalchemy>=2.0.32	Installed by uv sync --locked
alembic>=1.13	Installed by uv sync --locked
psycopg[binary]>=3.2	Installed by uv sync --locked
argon2-cffi>=23.1	Installed by uv sync --locked
cryptography>=43	Installed by uv sync --locked
python-multipart>=0.0.9	Installed by uv sync --locked
pymupdf>=1.24.10,<2	Installed by uv sync --locked
pytesseract>=0.3.13	Installed by uv sync --locked
pillow>=10.4	Installed by uv sync --locked
opencv-python-headless>=4.10	Installed by uv sync --locked
numpy>=1.26	Installed by uv sync --locked
presidio-analyzer>=2.2.355	Installed by uv sync --locked
presidio-anonymizer>=2.2.355	Installed by uv sync --locked
spacy>=3.7	Installed by uv sync --locked
google-genai>=1.0	Installed by uv sync --locked
structlog>=24.4	Installed by uv sync --locked
opentelemetry-api>=1.27	Installed by uv sync --locked

Dependency / constraint	Installation
opentelemetry-sdk>=1.27	Installed by uv sync --locked
opentelemetry-exporter-otlp-proto-http>=1.27	Installed by uv sync --locked
rapidfuzz>=3.9	Installed by uv sync --locked
en-core-web-sm	3.8.0 wheel from explosion/spacy-models on GitHub (URL pinned in pyproject.toml)

Backend development

Dependency / constraint	Installation
pytest>=8.3	Installed by uv sync --locked
pytest-timeout>=2.3	Installed by uv sync --locked
httpx>=0.27	Installed by uv sync --locked
reportlab>=4.2	Installed by uv sync --locked
ruff>=0.6	Installed by uv sync --locked

Dependency inventory: frontend

Direct constraints from package.json. npm ci uses package-lock.json for the exact resolved/transitive package graph. No manual per-package installation is necessary.

Frontend runtime

Package	Declared version
@fontsource/ibm-plex-sans	^5.3.0
@fontsource/ibm-plex-serif	^5.3.0
@hookform/resolvers	^3.10.0
@radix-ui/react-checkbox	^1.4.1
@radix-ui/react-dialog	^1.3.1
@radix-ui/react-dropdown-menu	^2.2.1
@radix-ui/react-label	^2.1.17
@radix-ui/react-progress	^1.2.1
@radix-ui/react-select	^2.4.1
@radix-ui/react-separator	^1.2.0
@radix-ui/react-slot	^1.5.0
@radix-ui/react-tabs	^1.2.1
@radix-ui/react-tooltip	^1.4.1
@tanstack/react-query	^5.107.1
class-variance-authority	^0.7.1
clsx	^2.1.1
lucide-react	^1.70.0
react	^19.1.0
react-dom	^19.1.0
react-hook-form	^7.91.0
react-router-dom	^7.18.5
sonner	^2.0.8

Package	Declared version
tailwind-merge	^3.7.0
zod	^3.25.76

Frontend development

Package	Declared version
@eslint/js	^9.25.0
@playwright/test	^1.65.0
@tailwindcss/vite	^4.3.5
@testing-library/jest-dom	^7.0.1
@testing-library/react	^16.3.3
@testing-library/user-event	^14.6.8
@types/node	^26.6.6
@types/react	^19.1.2
@types/react-dom	^19.1.2
@vitejs/plugin-react	^4.4.1
eslint	^9.25.0
eslint-plugin-react-hooks	^5.2.0
eslint-plugin-react-refresh	^0.4.19
globals	^16.0.0
jsdom	^31.0.0
tailwindcss	^4.3.5
tw-animate-css	^1.4.0
typescript	~5.8.3
typescript-eslint	^8.30.1
vite	^6.3.5
vitest	^3.2.7

GitHub handoff and source navigation

Share the project, not local runtime data

- Commit source, README, documentation, package manifests and lockfiles. Include this report if desired.

- Exclude backend/.env, API keys/passwords, backend/var, local databases, .venv, node_modules and test outputs.

- The reviewer creates a new environment file, independent secrets, API key, database and administrator.

- Use only clearly synthetic repository fixtures. Do not commit uploaded applicant documents or private screenshots.

- Review staged files before pushing. Git ignore rules cannot undo a secret that is already in Git history.

- No license file is currently declared; choose an appropriate license before describing the repository as licensed open-source.

Where to look in the code

Path	Review focus
backend/app/cli.py	Preflight, migrations, secret generation, bootstrap admin and worker commands.
backend/app/api/	Applicant submissions, staff decisions, access-controlled document routes and admin APIs.
backend/app/pipeline/	OCR, PII/tokenization, Gemini schema, evidence validation, name/PAN rules.
backend/app/models/ and services/	Persistent entities, view serialization, identity, policy, notifications and retention.
frontend/src/pages/	Applicant dashboard and staff overview, queue, review and admin UI.
frontend/src/components/	Shared stamps, notification bell, previews, uploader and timeline.
backend/tests/ and frontend/src/test/	Reproducible backend/frontend verification.
README.md and docs/DEPENDENCIES.md	Primary setup instructions and complete direct package inventory.

Rebuild this PDF and the dependency inventory from backend: uv run python ../scripts/build_project_report.py. The generator requires ReportLab, already included in the backend development dependency group.